

4.3 Romberg Integration

We can view Romberg's method as the natural generalization of the routine `qsimp` in the last section to integration schemes that are of higher order than Simpson's rule. The basic idea is to use the results from k successive refinements of the extended trapezoidal rule (implemented in `trapzd`) to remove all terms in the error series up to but not including $O(1/N^{2k})$. The routine `qsimp` is the case of $k = 2$. This is one example of a very general idea that goes by the name of *Richardson's deferred approach to the limit*: Perform some numerical algorithm for various values of a parameter h , and then extrapolate the result to the continuum limit $h = 0$.

Equation (4.2.4), which subtracts off the leading error term, is a special case of polynomial extrapolation. In the more general Romberg case, we can use Neville's algorithm (see §3.2) to extrapolate the successive refinements to zero stepsize. Neville's algorithm can in fact be coded very concisely within a Romberg integration routine. For clarity of the program, however, it seems better to do the extrapolation by a function call to `Poly_interp::rawinterp`, as given in §3.2.

```
romberg.h  template <class T>
           Doub qromb(T &func, Doub a, Doub b, const Doub eps=1.0e-10) {
Returns the integral of the function or functor func from a to b. Integration is performed by
Romberg's method of order 2K, where, e.g., K=2 is Simpson's rule.
           const Int JMAX=20, JMAXP=JMAX+1, K=5;
           Here EPS is the fractional accuracy desired, as determined by the extrapolation error estimate;
           JMAX limits the total number of steps; K is the number of points used in the extrapolation.
           VecDoub s(JMAX),h(JMAXP);           These store the successive trapezoidal approxi-
           Poly_interp polint(h,s,K);           mations and their relative stepsizes.
           h[0]=1.0;
           Trapzd<T> t(func,a,b);
           for (Int j=1;j<=JMAX;j++) {
               s[j-1]=t.next();
               if (j >= K) {
                   Doub ss=polint.rawinterp(j-K,0.0);
                   if (abs(polint.dy) <= eps*abs(ss)) return ss;
               }
               h[j]=0.25*h[j-1];
               This is a key step: The factor is 0.25 even though the stepsize is decreased by only
               0.5. This makes the extrapolation a polynomial in  $h^2$  as allowed by equation (4.2.1),
               not just a polynomial in  $h$ .
           }
           throw("Too many steps in routine qromb");
}
```

The routine `qromb` is quite powerful for sufficiently smooth (e.g., analytic) integrands, integrated over intervals that contain no singularities, and where the end-points are also nonsingular. `qromb`, in such circumstances, takes many, *many* fewer function evaluations than either of the routines in §4.2. For example, the integral

$$\int_0^2 x^4 \log(x + \sqrt{x^2 + 1}) dx$$

converges (with parameters as shown above) on the second extrapolation, after just 6 calls to `trapzd`, while `qsimp` requires 11 calls (32 times as many evaluations of the integrand) and `qtrap` requires 19 calls (8192 times as many evaluations of the integrand).